The Journal of Open Source Software

FluxOptics.jl: A Differentiable Wave Optics Framework for Inverse Design in Julia

Nicolas Barré [1]

1 Independent Researcher, France

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Abhishek Tiwari

Submitted: 03 November 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

FluxOptics.jl is a Julia package for simulating optical field propagation with full support for automatic differentiation. It enables gradient-based inverse design of optical components, which consists in determining the structure of an optical element (lens, diffraction grating, phase mask) that produces a desired light pattern or functionality. The package implements scalar wave propagation methods that are computationally efficient alternatives to finite-difference time-domain (FDTD) simulations, particularly suited for applications in additive manufacturing and low-cost optical characterization.

FluxOptics.jl provides multiple propagation algorithms for free-space and graded-index media, a composable architecture for building complex optical systems, GPU acceleration, and optimization tools including proximal operators for constrained inverse design. The architecture supports current scalar field applications and is designed to extend to vector field propagation for polarization-dependent components and dielectric metasurfaces.

Statement of Need

Inverse design of optical components has become increasingly important with the rise of freeform optics (Barré & Brunel, 2025; Schmidt et al., 2020), diffractive optical elements (DOEs) (Dinc et al., 2020), and metasurfaces (Molesky et al., 2018; Peurifoy et al., 2018). Traditional optimization approaches include gradient-free methods (evolutionary algorithms, Bayesian optimization, stochastic search) which can be effective for low-dimensional problems with up to hundreds of parameters. However, these methods become intractable for spatially-resolved optical elements with thousands or millions of parameters. Gradient-based optimization using automatic differentiation (Hughes et al., 2019; Minkov et al., 2020) enables efficient convergence at computational cost comparable to a single forward simulation.

However, existing tools face several limitations. Full-wave electromagnetic solvers like FDTD provide high accuracy but are computationally prohibitive for optimization, often requiring hours per forward simulation and limited to 2D or small 3D domains (Oskooi et al., 2010). Python packages like TorchOptics (Filipovich & Lvovsky, 2024) provide differentiable scalar wave propagation but suffer from performance bottlenecks.

FluxOptics.jl addresses these gaps through several key innovations. First, it provides high-performance differentiable propagation via Zygote.jl (Innes et al., 2019) or Enzyme.jl (Moses et al., 2021). Implemented in Julia (Bezanson et al., 2017), the package achieves CPU and GPU implementations that significantly outperform existing Python-based tools.

Second, the package provides an extensible component architecture designed for users to implement their own optical components. The interface offers two implementation patterns depending on performance requirements: rapid prototyping with automatic adjoint derivation

via automatic differentiation, or fine-grained control over memory allocations and gradient computation for production-level performance.

Third, FluxOptics.jl emphasizes composability. Optical systems are built using Julia's pipe operator (`|>`), allowing intuitive construction of cascaded systems. The `FieldProbe` mechanism enables capturing intermediate field states for multi-objective optimization, visualization, and debugging.

Fourth, the package implements efficient propagation methods through kernel caching strategies that avoid redundant computations. This makes scalar wave propagation practical for iterative optimization in applications such as additive manufacturing and intensity-based waveguide tomography.

Finally, the architecture is designed for extensibility from scalar to vector field propagation, with planned support for polarization-dependent components.

FluxOptics.jl emerged from practical research challenges in laser cavity design (Barré et al., 2014), waveguide characterization (Barré et al., 2021), and multimode beam control (Barré & Jesacher, 2022b, 2022a). The package provides a unified framework consolidating these diverse applications into a single, well-tested library with consistent API design.

Key Features

Component Architecture and Automatic Differentiation

All optical components inherit from a unified abstract type hierarchy that enables automatic differentiation. Components are divided into sources (generating optical fields) and pipe components (transforming fields). Connecting components with the pipe operator (`|>`) creates an `OpticalSystem` that executes the complete optical simulation when invoked.

The architecture offers two implementation patterns: **Pure components** require only a pure `propagate` method where automatic differentiation (Zygote or Enzyme) automatically derives adjoints, enabling rapid prototyping. **Custom components** implement the full interface with manual adjoint specification, providing fine-grained control over memory allocation and computational efficiency.

The package provides optimization tools built on Optimisers.jl including proximal operators and FISTA acceleration (Beck & Teboulle, 2009).

Optical Components and Field Propagation

FluxOptics.jl provides optical components including sources, modulators, and propagation methods. **Free-space propagation** includes Angular Spectrum Method, Rayleigh-Sommerfeld diffraction, and Collins integral for ABCD systems, with native support for tilted beam propagation. **Graded-index media** are handled through Beam Propagation Method with both paraxial and non-paraxial formulations. **Fourier optics** capabilities include Fourier lenses and frequency-domain filtering. **Active media** can be simulated using stationary gain sheets with saturable amplification.

System Composition

System composition uses pipe syntax (`source |> component1 |> component2 |> ...`) for intuitive construction. Sources can be `Trainable` or `Static`. Modulators include phase masks, amplitude masks, and Thin Element Approximation (TEA) diffractive elements. `FieldProbe` objects capture intermediate field states for multi-objective optimization and debugging.

Multi-Wavelength and Multimode Support

FluxOptics.jl natively supports polychromatic and multimode propagation.

GPU Acceleration

Seamless GPU acceleration via CUDA.jl with automatic context propagation through components.

Performance Benchmark

Comparison with TorchOptics (Filipovich & Lvovsky, 2024) on a beam splitter inverse design task (250×250 grid, 3 DOEs, 200 optimization iterations):

Platform	TorchOptics	FluxOptics.jl	Speedup
CPU (multi-threaded)	~7s	~5s	1.4×
GPU (NVIDIA RTX 4070 Super)	~3.5s	~0.27s	13×

CPU memory footprint: 41 MiB total allocation for 200 iterations (~205 KiB per iteration), enabling scaling to large multimode problems.

Tutorials and Documentation

FluxOptics.jl provides comprehensive documentation including five tutorials:

1. **Fox-Li Cavity Simulation**: Laser cavity eigenmodes in semi-degenerate resonators
2. **Field Retrieval from Intensity**: Reconstructing complex optical fields from intensity-only measurements, generalizing classical iterative projection methods (Fienup, 1982)
3. **Multi-Wavelength Beam Shaping**: Chromatic DOE cascades for independent control of red, green, and blue beams
4. **Waveguide Tomography**: Reconstructing refractive index profiles from angle-resolved intensity data
5. **Multimode Intensity Shaping**: Shaping 105 Laguerre-Gaussian modes using 2 cascaded DOEs with TV-norm regularization

Complete API documentation is available at https://anscoil.github.io/FluxOptics.jl/stable/.

Acknowledgments

There are no acknowledgments to declare and no conflicts of interest to disclose.

References

Barré, N., & Brunel, M. (2025). Differentiable wave propagation method for shape optimization of freeform optics beyond the paraxial approximation. *Optics Letters*, *50*(9), 2860–2863. https://doi.org/10.1364/OL.559067

Barré, N., & Jesacher, A. (2022a). Beam shaping of highly multimode sources with cascaded diffractive optical elements. *Procedia CIRP*, *111*, 566–570. https://doi.org/10.1016/j.procir.2022.08.151

Barré, N., & Jesacher, A. (2022b). Holographic beam shaping of partially coherent light. *Optics Letters*, *47*(2), 425–428. https://doi.org/10.1364/OL.444074

DRAFT

Barré, N., Romanelli, M., & Brunel, M. (2014). Role of cavity degeneracy for high-order mode excitation in end-pumped solid-state lasers. *Optics Letters*, *39*(4), 1022–1025. https://doi.org/10.1364/OL.39.001022

Barré, N., Shivaraman, R., Ackermann, L., Moser, S., Schmidt, M., Salter, P. S., Booth, M. J., & Jesacher, A. (2021). Tomographic refractive index profiling of direct laser written waveguides. *Optics Express*, *29*(22), 35414–35425. https://doi.org/10.1364/OE.434846

Beck, A., & Teboulle, M. (2009). A fast iterative shrinkage-thresholding algorithm for linear inverse problems. *SIAM Journal on Imaging Sciences*, *2*(1), 183–202. https://doi.org/10.1137/080716542

Bezanson, J., Edelman, A., Karpinski, S., & Shah, V. B. (2017). Julia: A fresh approach to numerical computing. *SIAM Review*, *59*(1), 65–98. https://doi.org/10.1137/141000671

Dinc, N. U., Lim, J., Kakkava, E., Moser, C., & Psaltis, D. (2020). Computer generated optical volume elements by additive manufacturing. *Nanophotonics*, *9*(13), 4173–4181. https://doi.org/10.1515/nanoph-2020-0196

Fienup, J. R. (1982). Phase retrieval algorithms: A comparison. *Applied Optics*, *21*(15), 2758–2769. https://doi.org/10.1364/AO.21.002758

Filipovich, M. J., & Lvovsky, A. I. (2024). *TorchOptics: An open-source python library for differentiable fourier optics simulations*. https://arxiv.org/abs/2411.18591

Hughes, T. W., Williamson, I. A., Minkov, M., & Fan, S. (2019). Forward-mode differentiation of maxwell's equations. *ACS Photonics*, *6*(11), 3010–3016. https://doi.org/10.1021/acsphotonics.9b01238

Innes, M., Edelman, A., Fischer, K., Rackauckas, C., Saba, E., Shah, V. B., & Tebbutt, W. (2019). A differentiable programming system to bridge machine learning and scientific computing. *arXiv Preprint arXiv:1907.07587*.

Minkov, M., Williamson, I. A., Andreani, L. C., Gerace, D., Lou, B., Song, A. Y., Hughes, T. W., & Fan, S. (2020). Inverse design of photonic crystals through automatic differentiation. *ACS Photonics*, *7*(7), 1729–1741. https://doi.org/10.1021/acsphotonics.0c00327

Molesky, S., Lin, Z., Piggott, A. Y., Jin, W., Vucković, J., & Rodriguez, A. W. (2018). Inverse design in nanophotonics. *Nature Photonics*, *12*(11), 659–670. https://doi.org/10.1038/s41566-018-0246-9

Moses, W. S., Churavy, V., Paehler, L., Hückelheim, J., Narayanan, S. H. K., Schanen, M., & Doerfert, J. (2021). Reverse-mode automatic differentiation and optimization of GPU kernels via enzyme. *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 1–16. https://doi.org/10.1145/3458817.3476165

Oskooi, A. F., Roundy, D., Ibanescu, M., Bermel, P., Joannopoulos, J., & Johnson, S. G. (2010). Meep: A flexible free-software package for electromagnetic simulations by the FDTD method. *Computer Physics Communications*, *181*(3), 687–702. https://doi.org/10.1016/j.cpc.2009.11.008

Peurifoy, J., Shen, Y., Jing, L., Yang, Y., Cano-Renteria, F., DeLacy, B. G., Joannopoulos, J. D., Tegmark, M., & Soljačić, M. (2018). Nanophotonic particle simulation and inverse design using artificial neural networks. *Science Advances*, *4*(6), eaar4206. https://doi.org/10.1126/sciadv.aar4206

Schmidt, S., Thiele, S., Toulouse, A., Bösel, C., Tiess, T., Herkommer, A., Gross, H., & Giessen, H. (2020). Tailored micro-optical freeform holograms for integrated complex beam shaping. *Optica*, *7*(10), 1279–1286. https://doi.org/10.1364/OPTICA.395177

DRAFT